

```
In [ ]: from itertools import product
        from pathlib import Path

        import polars as pl
        import marimo as mo
        from ortools.sat.python import cp_model
        from schemas import HorarioConfig, HorarioInstance, HorarioSolution

        DADOS = Path(__file__).parent / "dados"
        DADOS_V2 = Path(__file__).parent / "dados_v2"
```

Gerador de horário escolar — análise da resolução

Este documento explica como a implementação transforma os dados de uma escola num horário semanal válido e como o adapta quando os recursos mudam. A solução usa **Polars** para os dados tabulares, **Pydantic** para a sua validação, **CP-SAT do OR-Tools** para a resolução e **marimo** para apresentar código, fundamentação e resultados no mesmo relatório executável.

Há duas perguntas distintas: **o horário é válido?** e **qual dos horários válidos é preferível?** R1–R7 definem a validade, R8 exige a leitura dos CSV e O1 prefere horários com menos buracos para os professores. Quando os dados mudam, R9 dá prioridade a preservar as aulas do horário anterior.

Etapa	Elemento da implementação	Resultado
Importar e validar	<code>HorarioInstance.from_csv</code>	Dados estruturados da escola
Modelar	<code>HorarioSolver</code> e métodos <code>_r1_...</code> a <code>_r7_...</code>	Variáveis e restrições CP-SAT
Otimizar H0	<code>minimize_buracos()</code> e <code>solve()</code>	Horário inicial com o objetivo O1
Adaptar para H1	<code>incremental(h0)</code> e <code>solve()</code>	Horário que favorece a conservação de H0
Avaliar	Tabelas Polars e <code>contar_buracos</code>	Estados, tempos, alterações e buracos

As explicações seguintes descrevem o código existente. Os exemplos numéricos ilustram os CSV fornecidos; as decisões do solver continuam a ser calculadas a partir dos ficheiros, sem introduzir esses exemplos como dados do modelo.

1. Porque é que marimo é uma boa opção?

O trabalho pede um relatório que permita compreender e executar a solução. Aqui, cada explicação é uma célula `mo.md`, junto das células que produzem os resultados. Markdown organiza títulos, tabelas e código; as fórmulas usam LaTeX em strings `r"..."`, preservando as barras invertidas. [Documentação de Markdown do marimo](#).

Execução por dependências. O marimo analisa as variáveis definidas e usadas em cada célula e constrói um grafo de dependências. Ao executar uma célula alterada, atualiza as suas dependentes no modo automático. Por exemplo, a comparação incremental depende de `h0`; o horário apresentado a seguir depende de `h1`. Isto reduz resultados desatualizados por execução fora de ordem. Alterar um CSV no disco, por si só, não assegura uma atualização: é necessário voltar a executar a célula que o lê.

Cuidado com objetos mutáveis. O marimo não acompanha mutações internas, como adicionar restrições a `s0.model` noutra célula. Por isso, criar `s0`, aplicar O1 e resolver na mesma célula é uma boa decisão. H1 usa outro solver, criado e configurado na sua própria célula. [Modelo de execução do marimo](#).

Código reutilizável. `app.setup` reúne os imports e caminhos; `@app.class_definition` e `@app.function` permitem importar `HorarioSolver` e `contar_buracos` nos testes sem executar as células de demonstração. Esta organização aproxima o notebook de um módulo Python testável. [Reutilização de funções e classes](#).

Revisão e entrega. O notebook é guardado num ficheiro `.py`, conveniente para ler diferenças no Git, e pode ser apresentado ou exportado como relatório. [Perguntas frequentes do marimo](#). A reprodução completa também exige os CSV, `schemas.py`, o pacote local com as constantes e as dependências do projeto; o notebook não é autónomo.

2. Porque se escolheu Polars em vez de pandas?

Polars é uma escolha adequada para este projeto, porque a mesma biblioteca lê os CSV em `schemas.py` e apresenta os horários e as comparações neste notebook. Não é necessário acrescentar pandas para estas operações.

Aspeto	Vantagem de Polars	Aplicação nesta resolução
Colunas e tipos	Tipagem estrita, sem índice implícito	Campos explícitos do horário
Valores ausentes	<code>null</code> distinto de <code>NaN</code>	Distinguir ausência e valor numérico

Aspeto	Vantagem de Polars	Aplicação nesta resolução
Expressões	Operações sobre colunas	Filtros e agregações por turma ou professor
Paralelismo	Muitas operações em paralelo	Tratamento de dados maiores

Estas diferenças estão descritas no [guia oficial de migração de pandas para Polars](#). Tipagem tabular não substitui a validação das regras da escola: essa responsabilidade pertence aos modelos Pydantic e às restrições do solver.

Execução imediata e diferida. Polars também oferece `scan_csv` e `LazyFrame` : uma consulta pode ser planeada antes de `collect()` , com otimizações como antecipar filtros e selecionar apenas as colunas necessárias. [API lazy de Polars](#). A resolução atual usa `pl.read_csv(...).to_dicts()` , portanto materializa os dados imediatamente. Não utiliza um pipeline lazy entre os quatro CSV. Para ficheiros pequenos que serão integralmente convertidos em objetos Pydantic, esta opção mantém a leitura simples.

Eficiência e desempenho do Polars: O Polars, desenvolvido em Rust, foi concebido para processar dados de forma eficiente, recorrendo à execução paralela em múltiplas threads e a uma gestão otimizada da memória. Em comparação, o pandas, implementado em Python com componentes em C e Cython, executa muitas das suas operações numa única thread e pode apresentar custos adicionais associados à execução em Python. Estas características permitem ao Polars alcançar, em muitos cenários, tempos de execução inferiores e um menor consumo de memória, sobretudo no processamento de grandes volumes de dados. Contudo, o desempenho relativo depende das operações realizadas e das características dos dados.

A justificação mais sólida é, assim, a coerência da biblioteca de dados, a clareza das tabelas e a possibilidade de ampliar o tratamento tabular. Uma afirmação de superioridade de desempenho exigiria medições com os mesmos dados e operações nas duas bibliotecas.

3. Leitura dos CSV, validação e separação de responsabilidades — R8

`DADOS` e `DADOS_V2` são construídos com `Path(__file__).parent` . Assim, os ficheiros são procurados relativamente ao notebook, mesmo quando este é lançado a partir de outra pasta. Os caminhos identificam os conjuntos de entrada; o conteúdo escolar é lido de cada CSV.

Em `schemas.py` , `HorarioInstance.from_csv(pasta)` define uma função local `ler(nome)` , que lê o CSV com Polars e converte as linhas em dicionários. Essa função evita repetir a lógica de leitura para turmas, disciplinas, salas e exceções. Os

dicionários alimentam `HorarioInstance`, que agrega os objetos necessários ao solver.

- `empty_string_is_null=False` preserva strings vazias. Na coluna textual `sala_especial`, `' '` significa que a disciplina usa uma sala normal; isso corresponde à expressão `d.sala_especial or normal` em R7.
- `encoding="utf8-lossy"` permite substituir bytes UTF-8 inválidos por um carácter de substituição. É tolerante, mas pode alterar identificadores; uma leitura UTF-8 estrita seria preferível se a prioridade fosse rejeitar qualquer corrupção dos nomes.
- `to_dicts()` cria a ponte entre a tabela e os modelos Pydantic. É prática nesta escala, embora acrescente objetos Python e consumo de memória.

Pydantic valida os campos ao construir os modelos; isto é **validação em execução**, não apenas uma anotação para um verificador estático. [Modelos Pydantic](#).

`Field(ge=0)` rejeita cargas e quantidades negativas; `Literal` restringe `duplo_periodo` a `sim / nao` e `tipo` a `normal / especial`.

Esta validação ainda não cobre todas as relações: não garante nomes únicos, referências de salas existentes, períodos válidos nas exceções ou cargas pares em disciplinas duplas. Os dias e períodos vêm das constantes do pacote local, adequadas ao horizonte de cinco dias e cinco tempos do enunciado; não são lidos dos CSV.

Separar os dados do solver permite trocar CSV e testar outras instâncias sem reescrever as restrições. Também evita misturar a leitura dos ficheiros com a escolha de onde colocar cada aula.

4. Porque se modelou o problema com CP-SAT?

Colocar uma aula afeta simultaneamente a turma, o professor, a sala, a carga semanal e, por vezes, o bloco duplo. Escolher cada aula isoladamente pode conduzir a um impasse que exige rever escolhas anteriores. Declarar todas as restrições permite ao solver procurar uma combinação globalmente válida.

CP-SAT trabalha com variáveis inteiras e permite procurar soluções válidas ou otimizar um objetivo. As decisões binárias e as somas de tempos desta resolução encaixam nesse domínio. [Documentação do CP-SAT](#). Um algoritmo guloso ou uma pesquisa manual também seriam possíveis, mas obrigariam a implementar o tratamento de conflitos e retrocessos.

Variáveis de decisão

Sejam T as turmas, C as disciplinas, D os dias e P os períodos. Define-se uma variável para cada combinação:

$$x_{t,c,d,p} \in \{0, 1\}, \quad x_{t,c,d,p} = 1 \iff \text{a turma } t \text{ tem } c \text{ em } (d, p).$$

O dicionário `self.x` usa `(turma, disciplina, dia, periodo)` como chave. Isto torna os acessos legíveis e permite reutilizar a mesma representação nas restrições, nos hints e na lista de aulas da solução. Os nomes das variáveis ajudam a inspecionar o modelo; a chave do dicionário é o que identifica cada decisão no código.

O número inicial de variáveis é $|T| |C| |D| |P|$. Nos dados fornecidos, são $2 \times 6 \times 5 \times 5 = 300$, antes das variáveis auxiliares de O1. Não se cria uma dimensão para cada sala física: a capacidade é controlada por tipo de sala em R7.

Organização da classe

`__init__` guarda a instância e a configuração, cria o `CpModel`, declara as variáveis e chama `_build_model()`. Este método separa R1–R7 em métodos pequenos, facilitando relacionar o código com o enunciado. `CpSolver` é o motor que resolve esse modelo; não é o próprio modelo.

Geração das combinações de dias e períodos: A função `_slots()` devolve, em cada chamada, um novo iterador `product(dias, periodos)`, que representa o produto cartesiano entre os dias e os períodos, ou seja, todas as combinações possíveis entre ambos. Como este iterador se esgota após ser percorrido integralmente, a criação de um novo em cada chamada garante que cada restrição tem acesso à totalidade das combinações, evitando a reutilização de um iterador já esgotado por uma restrição anterior.

```
In [ ]: class HorarioSolver:
    """Modela R1–R7 e permite escolher O1 ou a preservação incremental de
    H0."""

    def __init__(self, instance: HorarioInstance, config: HorarioConfig |
    None = None):
        self.instance = instance
        self.config = config or HorarioConfig()
        self.model = cp_model.CpModel()
        self.x = {
            (t, d.disciplina, dia, p):
self.model.new_bool_var(f"x_{t}_{d.disciplina}_{dia}_{p}")
            for t in instance.turmas
            for d in instance.disciplinas
            for dia in instance.dias
            for p in instance.periodos
        }
        self._build_model()
        self.solver = cp_model.CpSolver()

    def _slots(self):
        return product(self.instance.dias, self.instance.periodos)

    def _build_model(self):
```

```

self._r1_uma_aula_por_tempo()
self._r2_carga_semanal()
self._r3_r4_uma_por_dia_e_duplo_periodo()
self._r5_professor_sem_sobreposicao()
self._r6_disponibilidade_professor()
self._r7_capacidade_salas()

def _r1_uma_aula_por_tempo(self):
    # R1 - Uma aula por turma em cada período
    for t in self.instance.turmas:
        for dia, p in self._slots():
            self.model.add_at_most_one(
                self.x[t, d.disciplina, dia, p] for d in
self.instance.disciplinas
            )

def _r2_carga_semanal(self):
    # R2 - Carga semanal exata
    for t in self.instance.turmas:
        for d in self.instance.disciplinas:
            self.model.add(
                sum(self.x[t, d.disciplina, dia, p] for dia, p in
self._slots())
                == d.carga_semanal
            )

def _r3_r4_uma_por_dia_e_duplo_periodo(self):
    for t in self.instance.turmas:
        for d in self.instance.disciplinas:
            dupla = d.duplo_periodo == "sim"
            for dia in self.instance.dias:
                # R3 - No máximo uma aula da mesma disciplina por dia,
por turma;
                # o bloco de 2 tempos de duplo período conta como uma
só ocorrência.
                self.model.add(
                    sum(self.x[t, d.disciplina, dia, p] for p in
self.instance.periodos)
                    <= (2 if dupla else 1)
                )
                # R4 - Duplo período só em blocos de 2 tempos
consecutivos no mesmo dia.
                if dupla:
                    for p in self.instance.periodos:
                        vizinhos = (self.x.get((t, d.disciplina, dia, p
+ k)) for k in (-1, 1))
                        self.model.add(
                            self.x[t, d.disciplina, dia, p]
                            <= sum(v for v in vizinhos if v is not
None)
                        )

def _r5_professor_sem_sobreposicao(self):
    # R5 - Um professor não pode dar duas aulas em simultâneo
    for prof in {d.professor for d in self.instance.disciplinas}:
        for dia, p in self._slots():

```

```

        self.model.add_at_most_one(
            self.x[t, d.disciplina, dia, p]
            for t in self.instance.turmas
            for d in self.instance.disciplinas
            if d.professor == prof
        )

    def _r6_disponibilidade_professor(self):
        # R6 - Um professor só pode dar aulas nos tempos em que está
        disponível
        for e in self.instance.excecoes:
            for d in self.instance.disciplinas:
                if d.professor == e.professor:
                    for t in self.instance.turmas:
                        self.model.add(self.x[t, d.disciplina, e.dia,
e.periodo] == 0)

    def _r7_capacidade_salas(self):
        # R7 - Nº de aulas em simultâneo numa sala não excede a quantidade
        dessa sala
        normal = next(s.sala for s in self.instance.salas if s.tipo ==
"normal")
        for s in self.instance.salas:
            disciplinas = [
                d.disciplina
                for d in self.instance.disciplinas
                if (d.sala_especial or normal) == s.sala
            ]
            for dia, p in self._slots():
                self.model.add(
                    sum(self.x[t, c, dia, p] for t in self.instance.turmas
for c in disciplinas)
                    <= s.quantidade
                )

    def incremental(self, h0: HorarioSolution):
        """Sugere H0 e minimiza aulas antigas não preservadas nas chaves
        atuais."""
        # R9 - parte de H0 (hint) e minimiza o nº de aulas que mudam de
        tempo
        anteriores = set(h0.aulas)
        for k, v in self.x.items():
            self.model.add_hint(v, k in anteriores)
        self.model.minimize(sum(1 - self.x[k] for k in anteriores if k in
self.x))
        self.solver.parameters.repair_hint = True

    def minimize_buracos(self):
        """Penaliza tempos livres entre a primeira e a última aula de cada
        dia."""
        # 01 - Minimiza a quantidade de aulas vazias entre a primeira e a
        última de cada professor
        gaps = []
        for prof in {d.professor for d in self.instance.disciplinas}:
            owned = [d.disciplina for d in self.instance.disciplinas if
d.professor == prof]

```

```

        for dia in self.instance.dias:
            busy = [
                sum(
                    self.x[turma, dis, dia, p]
                    for turma in self.instance.turmas
                    for dis in owned
                )
                for p in self.instance.periodos
            ]
            busy_size = len(busy)
            started = [self.model.new_bool_var(f"s_{prof}_{dia}_{k}")
            for k in range(busy_size)]
            continues = [
                self.model.new_bool_var(f"c_{prof}_{dia}_{k}") for k in
            range(busy_size)
            ]

            for k in range(busy_size):
                self.model.add_max_equality(
                    started[k], [busy[k]] + ([started[k - 1]] if k > 0
                else [])
                )
                self.model.add_max_equality(
                    continues[k], [busy[k]] + ([continues[k + 1]] if k
                < busy_size - 1 else [])
                )
                gap = self.model.new_bool_var(f"g_{prof}_{dia}_{k}")
                self.model.add(gap >= started[k] + continues[k] - 1 -
                busy[k])
                gaps.append(gap)

            self.model.minimize(sum(gaps))

    def solve(self, time_limit: float | None = None) -> HorarioSolution:
        """Resolve o modelo; extrai aulas apenas se existir uma solução
        válida."""
        self.solver.parameters.max_time_in_seconds = time_limit or
        self.config.time_limit
        self.solver.parameters.log_search_progress =
        self.config.log_search_progress
        status = self.solver.solve(self.model)
        if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return HorarioSolution(
                status=self.solver.status_name(status),
                tempo=self.solver.wall_time
            )
        return HorarioSolution(
            status=self.solver.status_name(status),
            tempo=self.solver.wall_time,
            aulas=[k for k, v in self.x.items() if self.solver.value(v)],
        )

```

5. Como se traduzem os requisitos em restrições?

R1 — uma turma não pode ter duas aulas no mesmo tempo

Para cada turma, dia e período:

$$\sum_{c \in C} x_{t,c,d,p} \leq 1.$$

`_r1_uma_aula_por_tempo()` usa `add_at_most_one`, que exprime diretamente esta cardinalidade sobre variáveis booleanas. Permite zero aulas naquele tempo: o enunciado não obriga a ocupar todos os períodos. Usar igualdade a um obrigaria a preencher a semana, alterando o problema.

R2 — cumprir exatamente a carga semanal

Se q_c é a carga da disciplina c , para cada turma:

$$\sum_{d \in D} \sum_{p \in P} x_{t,c,d,p} = q_c.$$

A igualdade impede tanto aulas em falta como aulas a mais. A carga é contada em **tempos**, pelo que um bloco duplo contribui com dois. Nos CSV iniciais, cada turma tem 17 tempos semanais distribuídos por 25 posições possíveis; a existência de períodos livres é esperada.

R1 e R2 complementam-se: a primeira evita sobreposições, enquanto a segunda obriga a colocar as aulas necessárias. Nenhuma delas, isoladamente, asseguraria ambas as propriedades.

R3 e R4 — distribuição diária e blocos duplos

Para uma disciplina normal, `_r3_r4_uma_por_dia_e_duplo_periodo()` impõe:

$$\sum_{p \in P} x_{t,c,d,p} \leq 1.$$

Isto distribui os tempos semanais por dias diferentes. Para uma disciplina dupla, o limite passa a dois tempos por dia. Esse limite, sozinho, ainda permitiria um tempo isolado ou dois tempos afastados.

Para cada período de uma disciplina dupla, acrescenta-se:

$$x_{t,c,d,p} \leq \sum_{q \in \{p-1, p+1\} \cap P} x_{t,c,d,q}.$$

Porque funciona esta combinação? Se um tempo está ocupado, pelo menos um vizinho tem de estar ocupado. Logo, um único tempo é impossível. Dois tempos não adjacentes também são impossíveis, pois nenhum tem um vizinho ocupado. Como o

limite diário impede três ou mais tempos, restam exatamente duas possibilidades: nenhuma aula ou um par consecutivo.

`self.x.get(...)` devolve `None` para vizinhos fora do horizonte. O teste `vis not None` exclui esses valores sem tentar converter uma variável simbólica do solver num booleano Python. No período 1 só existe o vizinho 2; no último período só existe o anterior.

Esta formulação evita variáveis adicionais para o início dos blocos. Depende, porém, do limite diário de dois: sem ele, a condição de vizinhança também aceitaria sequências de três tempos. Depende ainda de períodos numerados consecutivamente, como acontece nas constantes atuais.

Uma carga dupla de quatro tempos exige dois blocos em dias distintos. Uma carga dupla ímpar torna o modelo inviável, porque cada dia contribui com zero ou dois tempos. O código deixa essa inviabilidade ao solver; não a rejeita previamente em `schemas.py`.

R5 — ausência de sobreposições de professores

Seja C_f o conjunto de disciplinas do professor f . Em cada tempo:

$$\sum_{t \in T} \sum_{c \in C_f} x_{t,c,d,p} \leq 1.$$

O método agrupa **todas** as disciplinas do mesmo professor e percorre todas as turmas. Nos dados iniciais, a Prof. Diana dá História e Inglês: controlar cada disciplina separadamente permitiria indevidamente que desse ambas em simultâneo. O conjunto de nomes elimina professores repetidos na construção destas restrições.

R6 — disponibilidade por exceções

Por omissão, o professor está disponível. Para cada exceção (f, d, p) , todas as variáveis desse professor no tempo indicado ficam a zero:

$$x_{t,c,d,p} = 0 \quad (t \in T, c \in C_f).$$

A representação é compacta quando há poucas indisponibilidades. O mesmo método trata o novo CSV, sem assumir que só a Prof. Ana pode mudar. Uma exceção com dia ou período inexistente pode provocar um erro no acesso a `self.x`; validar esse domínio antes de construir o modelo seria útil.

R7 — capacidade dos tipos de sala

O código identifica a sala normal com `next(...)`. A expressão `d.sala_especial or normal` associa cada disciplina ao recurso necessário. Se C_r reúne as disciplinas

que usam o tipo de sala r :

$$\sum_{t \in T} \sum_{c \in C_r} x_{t,c,d,p} \leq \text{quantidade}_r.$$

Com um Laboratório, duas aulas que o exigem não podem ocorrer em simultâneo, mesmo com turmas e professores diferentes. A mesma lógica controla o Ginásio e as salas normais, e uma quantidade zero impede o uso desse recurso.

Pressupostos. Há um único grupo de salas normais, as salas do mesmo grupo são intercambiáveis e cada aula ocupa uma sala durante um tempo. O horário não atribui números a salas físicas. Sob estes pressupostos, o limite de capacidade permite uma atribuição de salas em cada tempo. Se fosse necessário manter uma sala específica, essa decisão teria de ser modelada explicitamente. A indisponibilidade de uma sala num tempo específico também exigiria dados e restrições de capacidade por tempo; o CSV atual representa uma capacidade constante durante toda a semana.

6. O1 — como se minimizam os buracos dos professores?

Um buraco é um tempo livre **entre** a primeira e a última aula do professor no mesmo dia. Tempos antes da primeira aula e depois da última não contam. A contagem é por professor, agregando as suas turmas e disciplinas.

Períodos com aulas	Buracos	Razão
1, 3, 4	1	O período 2 está entre aulas
2, 5	2	Os períodos 3 e 4 estão entre aulas
3, 4	0	Não se penalizam os períodos antes e depois
Nenhum, ou apenas um	0	Não há intervalo livre entre duas aulas

Ocupação e propagação nos dois sentidos

Para um professor e um dia, `busy[k]` soma as suas aulas no período k . Graças a R5, essa soma só pode valer zero ou um, mesmo sendo uma expressão linear e não uma nova variável booleana.

`started[k]` indica se existe uma aula até esse período; `continues[k]` indica se existe uma aula desse período até ao final do dia:

$$s_k = \max(b_k, s_{k-1}), \quad c_k = \max(b_k, c_{k+1}).$$

Propagação da existência de aulas: Nos extremos, considera-se apenas b_k . As chamadas a `add_max_equality` impõem estas relações independentemente da

ordem em que as restrições são adicionadas. A variável `started` indica a existência de pelo menos uma aula desde o início até ao período considerado, enquanto `continues` indica a existência de pelo menos uma aula desde esse período até ao fim. Estas variáveis não indicam necessariamente que o professor esteja a lecionar no período em questão. Embora seja possível formular relações equivalentes através de restrições lineares, a utilização de `add_max_equality` simplifica o código. A eventual diferença de desempenho entre as duas abordagens deverá ser avaliada empiricamente, podendo ser pouco relevante face à dimensão reduzida do conjunto de dados.

Variáveis de penalização

Para cada período, cria-se `gap`, uma variável booleana, com:

$$g_k \geq s_k + c_k - 1 - b_k, \quad \min \sum_{f,d,k} g_{f,d,k}.$$

Se o tempo está livre e há aulas antes e depois, o lado direito vale um: `gap` é obrigatoriamente um. Se há aula, ou se falta uma aula num dos lados, o lado direito não obriga a penalizar e a minimização favorece zero. Nos cinco professores, cinco dias e cinco períodos iniciais, criam-se 375 variáveis auxiliares: `started`, `continues` e `gap` por posição.

Pormenor de precisão. A desigualdade impõe apenas um limite inferior. Num ótimo, todos os `gap` desnecessários valem zero, pois poderiam ser reduzidos sem afetar o horário. Numa solução apenas `FEASIBLE`, o modelo não garante essa equivalência exata para todos os auxiliares. Por isso, contar os buracos a partir das aulas é a forma direta de avaliar o horário realmente devolvido.

7. Horário inicial H0 e interpretação da resolução

A próxima célula lê `dados/`, constrói R1–R7, acrescenta O1 e resolve. Os passos estão separados na API para permitir reutilizar o mesmo modelo de restrições com objetivos diferentes. `HorarioConfig` define, por omissão, 30 segundos de limite para cada chamada ao solver.

`solve()` configura o limite e o registo da pesquisa, chama o CP-SAT e extrai as chaves de `self.x` cujas variáveis valem um. A solução guarda o estado, o tempo do solver e as aulas; não guarda uma tabela nem salas físicas. A expressão `time_limit` or `self.config.time_limit` usa a configuração também quando é passado zero; não valida limites negativos.

Estado	Como interpretar
<code>OPTIMAL</code>	Solução válida; com objetivo, a otimalidade foi demonstrada

Estado	Como interpretar
FEASIBLE	Solução válida, sem prova de otimalidade
INFEASIBLE	Foi demonstrado que as restrições não admitem solução
UNKNOWN	A execução terminou sem uma solução nem uma prova de inviabilidade
MODEL_INVALID	O modelo foi rejeitado por ser inválido

Sem objetivo, `OPTIMAL` não significa que o horário seja melhor em O1. Estes estados seguem a [documentação do CP-SAT](#).

O código só extrai aulas nos estados válidos. Nos restantes, devolve uma lista vazia: uma tabela vazia não representa um horário válido com zero buracos. As métricas devem ser interpretadas apenas quando existe solução.

`pl.DataFrame(..., orient="row")` transforma cada tuplo numa linha e `schema=[...]` identifica as quatro colunas. A apresentação é uma lista de aulas ocupadas; os períodos livres não aparecem como linhas.

```
In [ ]: # H0 - horário principal: R1-R8 + O1 (minimizar buracos)
s0 = HorarioSolver(HorarioInstance.from_csv(DADOS))
s0.minimize_buracos()
h0 = s0.solve()
pl.DataFrame(h0.aulas, schema=["turma", "disciplina", "dia", "periodo"],
orient="row")
```

8. R9 — adaptação incremental de H0 para H1

Em `dados_v2/`, a Prof. Ana passa a estar indisponível à sexta-feira nos períodos 4 e 5. As restantes regras continuam em vigor. Reutilizar H0 diretamente poderia violar a nova disponibilidade; gerar um horário sem referência a H0 poderia alterar muitas aulas que continuam válidas.

Duas componentes com funções diferentes

Hint. `anteriores = set(h0.aulas)` reúne as posições ocupadas. Para cada variável do novo modelo, `add_hint(v, k in anteriores)` sugere um se a posição estava ocupada e zero caso contrário. Isto fornece um ponto de partida à pesquisa; não acrescenta uma obrigação `x[k] == 1`. `repair_hint=True` configura uma tentativa de reparar o hint quando já não satisfaz os novos dados. O solver pode alterar qualquer aula se for necessário para satisfazer as restrições.

Objetivo de estabilidade. Seja A_0 o conjunto das aulas de H0 cujas chaves também existem no novo modelo:

$$\min \sum_{k \in A_0} (1 - x_k).$$

Cada aula preservada contribui com zero; cada posição antiga abandonada contribui com um. O hint orienta a pesquisa, enquanto este objetivo favorece explicitamente horários que conservam mais aulas.

Com as mesmas turmas, disciplinas e cargas, mover uma aula elimina uma posição antiga e acrescenta uma nova. Contar só a posição perdida evita contar a mesma mudança duas vezes. Para os blocos duplos, a unidade contada continua a ser o **tempo letivo**, não o bloco.

Não se fixam rigidamente as aulas não afetadas: uma indisponibilidade pode obrigar a deslocções em cadeia por conflitos de turma, professor ou sala. Permitir essa reorganização é mais flexível. Também não se reutiliza o estado interno da pesquisa anterior: cria-se um novo modelo, aproveitando a solução de H0 como hint e referência do objetivo.

Alcance da métrica e do objetivo

`mudancas(h1)` calcula `len(set(h0.aulas) - set(h1.aulas))`. Mede tempos antigos não preservados, sem identificar individualmente aulas da mesma disciplina. Não mede alterações de professor quando a chave se mantém, nem mudanças de sala física, que não está na chave. Aulas de uma turma nova não são penalizadas por não terem posição em H0. Se uma turma ou disciplina desaparecer, a métrica inclui essas aulas antigas, mas o objetivo exclui as chaves ausentes do novo modelo: são perdas inevitáveis, sem variável atual para as preservar.

H0 otimiza buracos; H1 otimiza alterações. Chamar `minimize_buracos()` e depois `incremental()` no mesmo modelo não combina os objetivos: a última chamada a `minimize` substitui a anterior. Esta resolução usa solvers distintos, de acordo com o enunciado, que dispensa a otimalidade de O1 em H1. Uma prioridade adicional aos buracos em H1 exigiria uma otimização lexicográfica ou uma função ponderada devidamente justificada.

Como ler a comparação seguinte

O ramo **do zero** procura um horário válido com os novos dados, sem hint e sem objetivo de estabilidade. O ramo **incremental** recebe o hint e minimiza alterações. A comparação mostra o efeito conjunto dessas duas escolhas; não isola o efeito do hint.

Menos alterações e menor tempo são propriedades diferentes. O objetivo favorece a primeira; o hint não garante a segunda. Otimizar e provar otimalidade pode até demorar mais do que encontrar qualquer solução válida. Devem ser usados os valores

efetivamente observados na tabela, incluindo os estados, em vez de assumir que o incremental é sempre mais rápido.

```
In [ ]: # R9 - H1 do zero (sem H0) vs. incremental (hint de H0 + minimizar
        mudanças)

def mudancas(h1):
    return len(set(h0.aulas) - set(h1.aulas))

zero = HorarioSolver(HorarioInstance.from_csv(DADOS_V2)).solve()
s1 = HorarioSolver(HorarioInstance.from_csv(DADOS_V2))
s1.incremental(h0)
h1 = s1.solve()
pl.DataFrame(
    {
        "abordagem": ["do zero", "incremental"],
        "estado": [zero.status, h1.status],
        "tempo (s)": [zero.tempo, h1.tempo],
        "aulas alteradas": [mudancas(zero), mudancas(h1)],
    }
)
```

Horário adaptado H1

A tabela seguinte apresenta as aulas da solução incremental, com os mesmos campos de H0. Para avaliar a adaptação, importa confirmar que o estado anterior é `OPTIMAL` ou `FEASIBLE` e que as aulas respeitam os novos dados. Uma quantidade baixa de alterações só tem significado num horário válido.

`tempo` é `solver.wall_time`, em segundos. Não inclui a leitura dos CSV, a criação das variáveis, a construção das restrições ou a apresentação. Como H1 usa um modelo novo, esses custos continuam a existir. Para avaliar o fluxo completo seria necessário medir também esse intervalo, por exemplo com `time.perf_counter()` antes da leitura e depois de `solve()`.

Uma execução é uma ilustração, não um estudo de desempenho. Uma avaliação mais forte repetiria os cenários, compararia medianas e dispersão e registaria hardware, versões, parâmetros, limites de tempo e estados. Para isolar o hint, os dois ramos teriam de usar o mesmo objetivo de estabilidade, diferindo apenas no fornecimento do hint.

```
In [ ]: pl.DataFrame(h1.aulas, schema=["turma", "disciplina", "dia", "periodo"],
                    orient="row")
```

9. Avaliação independente dos buracos

`contar_buracos(instance, aulas)` avalia o horário concreto, sem consultar `started`, `continues` ou `gap`. Primeiro associa cada disciplina ao seu professor;

depois reúne os períodos por `(professor, dia)`.

Para um grupo não vazio de períodos S , a contribuição é:

$$\text{buracos}(S) = \max(S) - \min(S) + 1 - |S|.$$

O primeiro termo é o número de posições entre a primeira e a última aula, incluindo os extremos. Retirar o número de períodos ocupados deixa apenas as posições livres no interior. Por exemplo, para $S = \{2, 5\}$, o intervalo tem quatro posições e duas aulas: há dois buracos.

A fórmula dispensa ordenar os períodos ou percorrer os tempos livres. Um dia sem aulas não cria um grupo, contribuindo implicitamente com zero; uma única aula produz $1 - 1 = 0$. Somam-se todos os professores e dias.

Pressupostos da contagem: não há sobreposições do mesmo professor, as aulas não estão duplicadas e os períodos são inteiros consecutivos. R5 assegura o primeiro pressuposto para as soluções do solver. Se a função recebesse aulas arbitrárias com períodos repetidos, `len(ps)` poderia dar uma contagem incorreta; a função é uma métrica, não um validador completo.

Este cálculo é independente da formulação de O1. É útil para comparar horários e verificar se o valor do objetivo corresponde aos buracos reais, com a ressalva sobre auxiliares em soluções apenas `FEASIBLE`.

```
In [ ]: def contar_buracos(instance, aulas):  
        """Conta tempos livres interiores por professor e dia num horário  
        válido."""  
        prof = {d.disciplina: d.professor for d in instance.disciplinas}  
        tempos = {}  
        for _, c, dia, p in aulas:  
            tempos.setdefault((prof[c], dia), []).append(p)  
        return sum(max(ps) - min(ps) + 1 - len(ps) for ps in tempos.values())
```

Comparação de um horário sem O1 com H0

A próxima célula cria outro solver com os dados iniciais, mantendo R1–R7 e sem acrescentar o objetivo O1. Compara-o com H0 usando a mesma função

`contar_buracos` nos dois casos. Assim, a comparação incide sobre as aulas realmente devolvidas, e não só sobre variáveis auxiliares.

Se H0 tiver estado `OPTIMAL`, minimiza os buracos neste modelo. Um valor zero demonstra que não existe solução melhor para O1, pois a contagem nunca pode ser negativa num horário válido. Se tiver estado `FEASIBLE`, é necessário observar a contagem: existe uma solução, mas não uma prova geral de que seja a melhor. O horário sem O1 pode, por acaso, já ter poucos buracos; o objetivo é que orienta explicitamente a pesquisa para os reduzir.

A célula faz uma execução de cada abordagem. Também aqui os tempos são apenas os do solver e as métricas só são interpretáveis com soluções válidas.

```
In [ ]: # 01 - buracos sem objetivo vs. com 01 (H0)
sem_o1 = HorarioSolver(s0.instance).solve()
pl.DataFrame(
    {
        "abordagem": ["sem 01", "com 01"],
        "estado": [sem_o1.status, h0.status],
        "tempo (s)": [sem_o1.tempo, h0.tempo],
        "buracos": [
            contar_buracos(s0.instance, sem_o1.aulas),
            contar_buracos(s0.instance, h0.aulas),
        ],
    }
)
```

10. Como se valida a resolução?

Os testes em `tests/test_horario_escolar.py` e as funções de `tests/conftest.py` cobrem as regras por duas vias: verificam horários produzidos e forçam casos que deveriam ser impossíveis.

Aspeto	Evidência implementada nos testes
Dimensão das variáveis	Número de variáveis igual ao produto das dimensões
R1-R7	Verificador independente sobre as aulas da solução
R1 e R2	Sobreposição de uma turma e carga semanal excedida tornam o modelo inviável
R3 e R4	Repetição diária, tempo duplo isolado e par não consecutivo são rejeitados
R5 e R6	Sobreposições e indisponibilidades rejeitadas; tempo disponível aceite
R7	Falta de capacidade de salas normais e especiais é detetada
R8 e generalização	Leitura dos CSV e cenário com outra turma, disciplina e exceção
Validação de campos	Um valor inválido de <code>duplo_periodo</code> é rejeitado
R9	H1 é válido e, no cenário testado, perde apenas as posições antigas agora proibidas
O1	Exemplos de contagem, comparação de horários e caso com um buraco forçado

`_verificar` conta diretamente as aulas por turma, disciplina, professor e sala, sem chamar os métodos que construíram as restrições. Esta independência torna os testes

mais úteis para detetar erros de modelação. As assumptions fixam decisões para uma chamada ao solver e são limpas depois, permitindo testar conflitos sem acrescentar fixações permanentes.

O solver da fixture `o1` é separado: `minimize_buracos()` altera o modelo, pelo que partilhar esse objeto com testes sem objetivo confundiria os cenários. A igualdade entre objetivo e contagem de buracos é verificada no cenário de teste; devido à formulação dos auxiliares, não deve ser generalizada a qualquer incumbente apenas `FEASIBLE`.

Para executar os testes a partir da raiz do repositório, com o ambiente do projeto preparado:

```
uv run pytest tests/test_horario_escolar.py
```

Os testes mostram o comportamento nos casos cobertos. Não constituem uma prova de viabilidade de qualquer CSV futuro, nem um benchmark de rapidez incremental. A fundamentação das restrições e os testes complementam-se.